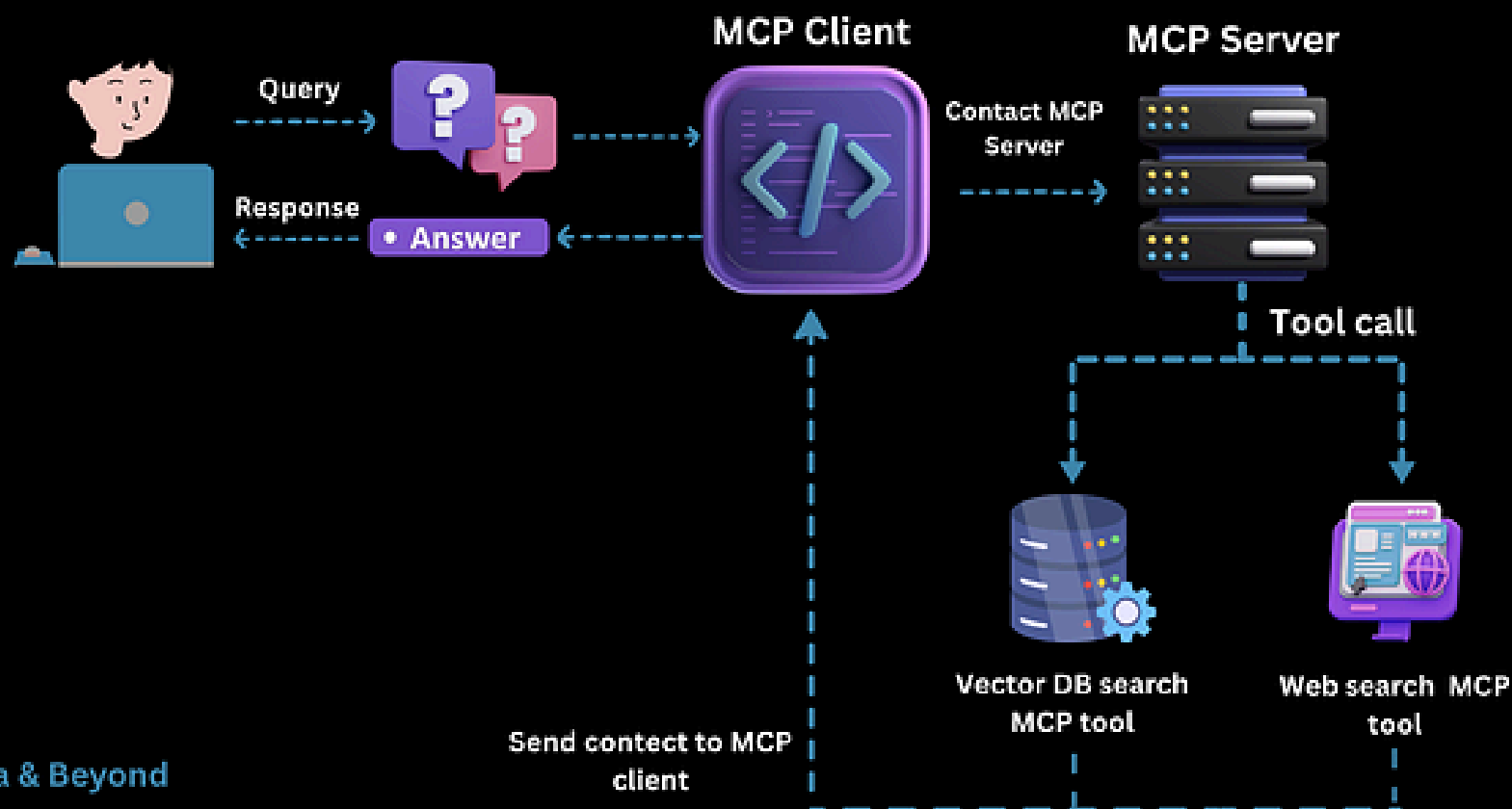




Building MCP Powered Agentic RAG Application: Step by Step Guide



July 16, 2025

Building MCP-Powered Agentic RAG Application: Step-by-Step Guide (2/2)

You can read the article for free from here



Youssef Hosni

14 min read

[Download article](#) 

Contents 6

1 5. Define MCP Tools

2 6. Building RAG Pipeline

3 7. Putting Everything Together and Testing the System

– If you like the article and would like to support me, make sure to:

– Subscribe to my newsletter, To Data & Beyond, to get full and early access to my articles:

[Show all 6 sections](#)

You can read the article for free from [here](#)🔗

While many tutorials show how to build Retrieval-Augmented Generation (RAG) systems by connecting a language model to a single database, this approach is often too limited.

This guide takes you to the next level with **Agentic RAG**, where you won't just build a system that retrieves information — you'll build one that intelligently decides *where* to look. It's the difference between an AI that can only read one assigned book and an AI that knows exactly which book to pull from the library.

To achieve this, we will use the **Model Context Protocol (MCP)**, a modern framework designed specifically for creating powerful, multi-tool AI agents. In this step-by-step tutorial, we will walk you through the entire process:

- Setting up a complete MCP server.
- Defining custom tools for both private data and live web searches.
- Constructing a full RAG pipeline that puts the agent in control.

By the end, you'll have a fully functional and intelligent application capable of fielding complex queries by dynamically sourcing the best possible context for a truly accurate answer.

You can read part 1 from here:

[Building MCP-Powered Agentic RAG Application: Step-by-Step Guide \(1/2\)](#) 🔗 *A Step-by-Step Guide to Building an Agentic RAG Application with MCP*

Building MCP Powered Agentic RAG Application: Step by Step Guide

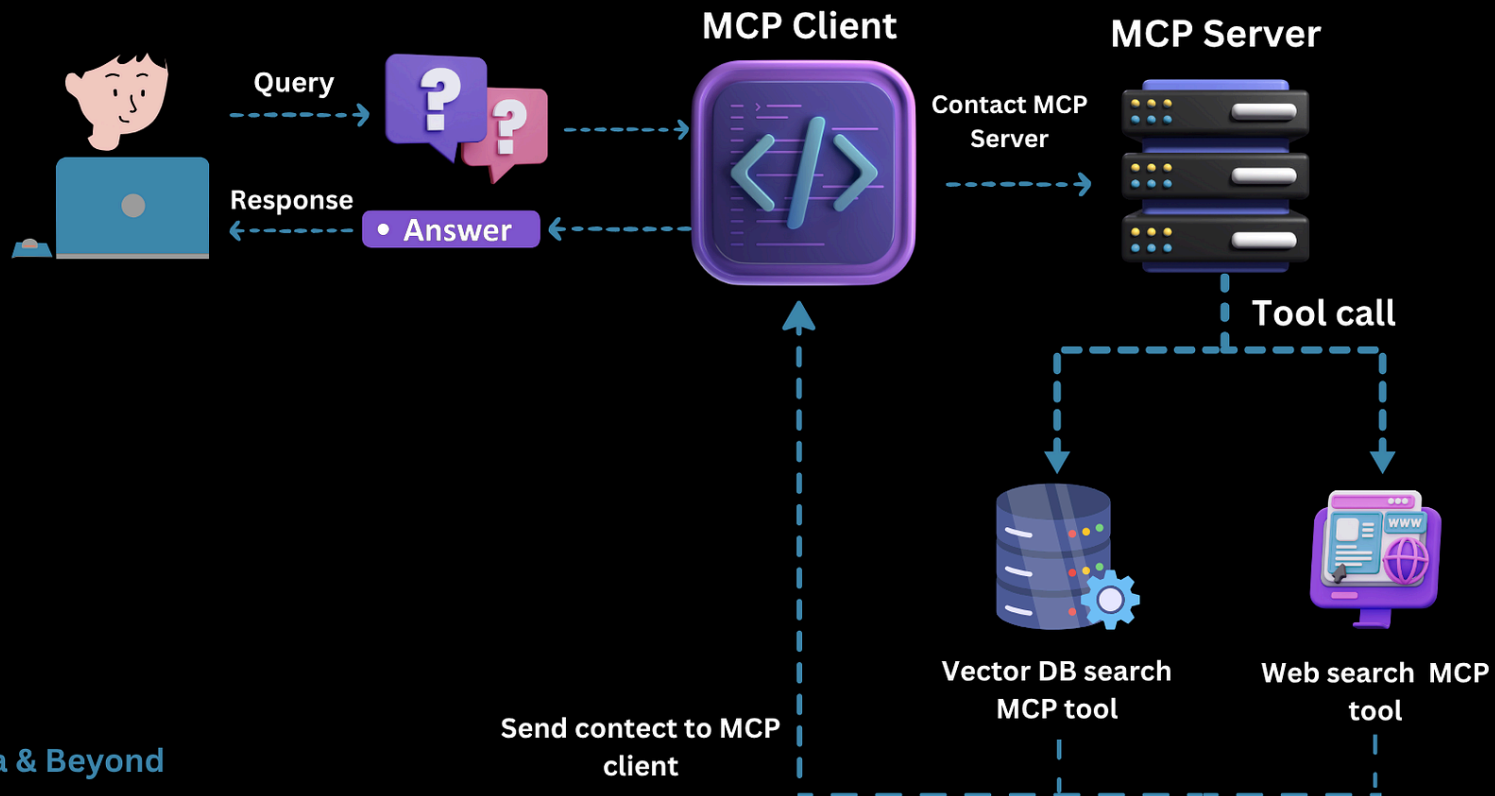


Table of Contents:

1. [Introduction to Model Context Protocol \(MCP\)](#) [↗] [Part 1]
2. [MCP-Powered Agentic RAG Architecture Overview](#) [↗] [Part 1]
3. [Setting Up the Working Environment](#) [↗] [Part 1]

4. [Set up an MCP server instance](#) [Part 1]
5. Define MCP Tools [Part 2]
6. Building RAG Pipeline [part 2]
7. Putting Everything Together and Testing the System [Part 2]

You can find all the codes in this [GitHub Repository](#)

Most insights I share in Medium have previously been shared in my weekly newsletter, To Data & Beyond.

If you want to be up-to-date with the frenetic world of AI while also feeling inspired to take action or, at the very least, to be well-prepared for the future ahead of us, this is for you.

 **Subscribe below**  *to become an AI leader among your peers and receive content not present in any other platform, including Medium:*

[To Data & Beyond](#) | [Youssef Hosni](#) | [Substack](#) *Data Science, Machine Learning, AI, and what is beyond them. Click to read To Data & Beyond, by Youssef Hosni, a...*

5. Define MCP Tools

Our **MCP server** is now instantiated, but it's like a skilled worker with an empty toolbox. To make it useful, we need to give it **tools**. In the MCP world, a tool is simply a Python function that we "register" with the server, making it a capability the agent can choose to use. This registration is done with a simple decorator: `@mcp_server.tool()`.

Crucially, the agent relies on the function's **docstring** to understand what the tool does and when it should be used. A well-written docstring is not just for developers; it's a direct instruction to the AI, guiding its decision-making process.

Let's start by defining our first tool: the one that connects to our private vector database. This tool will be responsible for searching our internal Python FAQ knowledge base.

Add the following code to your **mcp_server.py** file.

```
1 @mcp_server.tool()
2 def python_faq_retrieval_tool(query: str) → str:
3     """
4     Retrieve the most relevant documents from the Python FAQ collection.
5     Use this tool when the user asks about general Python programming concepts.
6
7     Args:
8         query (str): The user query to retrieve the most relevant documents.
9
10    Returns:
11        str: The most relevant documents retrieved from the vector DB.
12    """
13    if not isinstance(query, str):
14        raise TypeError("Query must be a string.")
15
16    # Use the single, pre-initialized faq_engine instance for efficiency
17    return faq_engine.answer_question(query)
```

Let's break down this powerful piece of code:

- **@mcp_server.tool()**: This is the magic decorator. It tells our mcp_server instance, "Hey, here is a new capability you can use!" The server automatically inspects the function's name, its parameters, and — most importantly — its docstring to learn about this new skill.

- **The Docstring:** This is the most critical part for the agentic behavior. When the agent receives a user query, it will read this description to decide if this tool is the right one for the job. We've explicitly told it to use this tool when the user asks about general Python programming concepts. This clear instruction is what allows the agent to distinguish between needing to search our internal FAQ versus, say, searching the web.
- **python_faq_retrieval_tool(query: str) → str:** This is a standard Python function with type hints. The MCP framework uses these hints to validate inputs and understand the data types the tool works with.
- **faq_engine.answer_question(query):** This is the line that executes the tool's core logic. It calls a method on an `faq_engine` object (which we will build in the "Building RAG Pipeline" section) to perform the actual vector search in our Qdrant database and return the results as a string.

With this single function, we've given our agent its first sense — the ability to look up information in its own specialized memory. Next, we'll give it the ability to look outward to the web.

Now, what if the user asks about a very recent Python library, a current event, or a topic not covered in our internal knowledge base? For that, our agent needs to be able to browse the internet.

We'll equip our agent with a second tool that uses the [FireCrawl API](#) to perform a live web search. This gives our agent a way to find real-time, public information, making it vastly more versatile.

Add the final tool to your `mcp_server.py`:

```
1 @mcp_server.tool()
2 def firecrawl_web_search_tool(query: str) → List[str]:
3     """
4     Search for information on a given topic using Firecrawl.
5     Use this tool when the user asks a specific question not related to the Python FAQ.
6
```

```
7     Args:
8         query (str): The user query to search for information.
9
10    Returns:
11        List[str]: A list of the most relevant web search results.
12    """
13    if not isinstance(query, str):
14        raise TypeError("Query must be a string.")
15
16    url = "https://api.firecrawl.dev/v1/search"
17    api_key = os.getenv('FIRECRAWL_API_KEY')
18
19    if not api_key:
20        return ["Error: FIRECRAWL_API_KEY environment variable is not set."]
21
22    payload = {"query": query, "timeout": 60000}
23    headers = {
24        "Authorization": f"Bearer {api_key}",
25        "Content-Type": "application/json"
26    }
27
28    try:
29        response = requests.post(url, json=payload, headers=headers)
30        response.raise_for_status() # Raise an exception for bad status codes (4xx or 5xx)
31        # Assuming the API returns JSON with a key like "data" or "results"
32        # Adjust .get("data", ...) if the key is different
33        return response.json().get("data", ["No results found from web search."])
34    except requests.exceptions.RequestException as e:
35        return [f"Error connecting to Firecrawl API: {e}"]
```


This `firecrawl_web_search_tool` function is another powerful capability for our agent. Registered with the same `@mcp_server.tool()` decorator, its docstring provides the critical instruction for when it should be used: for any question not related to the Python FAQ.

This creates a clear distinction from our first tool, enabling the agent to make an intelligent choice. The function's logic is straightforward: it securely retrieves the Firecrawl API key from our environment variables, constructs a POST request containing the user's query, and sends it to the Firecrawl API.

To ensure our application is resilient and doesn't crash on network failures, the entire API call is wrapped in a `try...except` block, allowing it to handle connection issues gracefully. Upon a successful request, it parses the JSON response and returns a list of search results, effectively giving our agent access to the vast information of the internet.

With both tools now defined, our agent has a choice: look inward into its private knowledge base or look outward to the public web. Next, we will build the engine that powers our internal knowledge search.

[Everything I've Written, One Button Away, With 40% Off](#)  *Announcing My Ebooks Bundle + 50% Discount to my Followers*

6. Building RAG Pipeline

Our `python_faq_retrieval_tool` is currently just a promise; it relies on an `faq_engine` object to do the heavy lifting of vector search, but we haven't built this engine yet. This component is the heart of the "Retrieval" part of our RAG system. It will be responsible for two critical tasks:

1. **Indexing:** Taking our plain text data, converting it into numerical representations (embeddings), and storing them efficiently in our Qdrant vector database.
2. **Searching:** Providing a method that can take a user's query, convert it into an embedding, and search the database for the most relevant pieces of stored information.

To accomplish this, we'll orchestrate several powerful libraries. We'll use llama-index for its robust embedding capabilities, qdrant-client to communicate with our database, and a few standard Python utilities to handle the data processing efficiently.

Let's begin by creating the **rag_app.py** file and adding the necessary imports that will form the backbone of our pipeline.

```
1 import uuid
2 from itertools import islice
3 from typing import List, Dict, Any, Generator
4
5 from llama_index.embeddings.huggingface import HuggingFaceEmbedding
6 from tqdm import tqdm
7 from qdrant_client import models, QdrantClient
```

The first step in building our pipeline is to define the knowledge base itself. This is the proprietary data our agent will search through to answer questions related to Python.

In our **rag_app.py** file, add our source text just below the imports:

```
1 PYTHON_FAQ_TEXT = """
2 Question: What is the difference between a list and a tuple in Python?
3 Answer: Lists are mutable, meaning their elements can be changed, while tuples are immutable. Lists use square brackets to create and access elements, while tuples use parentheses.
4
5 Question: What are Python decorators?
6 Answer: Decorators are a design pattern in Python that allows a user to add new functionality to an existing object without modifying its structure. They are often used to modify the behavior of functions or classes.
7
8 Question: How does Python's Global Interpreter Lock (GIL) work?
9 Answer: The GIL is a mutex that protects access to Python objects, preventing multiple threads from executing the Python interpreter at the same time. It is implemented in CPython and is not present in other Python implementations like PyPy or Jython.
```

```
10
11 Question: What is the difference between `==` and `is` in Python?
12 Answer: `==` checks for equality of value (do two objects have the same content?), while `is` checks for id
13
14 Question: Explain list comprehensions and their benefits.
15 Answer: List comprehensions provide a concise way to create lists. They are often more readable and faster
16
17 Question: What is `*args` and `**kwargs` in function definitions?
18 Answer: `*args` allows you to pass a variable number of non-keyword arguments to a function, which are rece
19
20 Question: What are Python's magic methods (e.g., `__init__`, `__str__`)?
21 Answer: Magic methods, or dunder methods, are special methods that you can define to add "magic" to your cl
22
23 Question: How does error handling work in Python?
24 Answer: Python uses `try...except` blocks to handle exceptions. Code that might raise an error is placed in
25
26 Question: What is the purpose of the `if __name__ == "__main__":` block?
27 Answer: This block ensures that the code inside it only runs when the script is executed directly, not when
28
29 Question: What are generators in Python?
30 Answer: Generators are a simple way to create iterators. They are functions that use the `yield` keyword to
31 """
```

For this guide, we're embedding our knowledge directly into the script as a multi-line string. In a production application, you would typically load this data from a collection of documents, like **text files**, **PDFs**, or a **database**, but the principle remains the same.

This **PYTHON_FAQ_TEXT** variable contains a curated list of common Python questions and answers. Each Q&A pair is a self-contained chunk of information, making it an ideal format for our vector database. When we index this data, each chunk will

become a searchable unit that our agent can retrieve.

With our data defined, the next step is to create a class that can process it.

With our data defined, we now need to build the engine that can process, index, and query it. This is where the FAQEngine class comes in. It's the central component of our RAG pipeline, encapsulating all the logic for interacting with both the embedding model and the Qdrant vector database.

First, we'll add a small helper function to our rag_app.py. This function will help us process our data in manageable chunks, which is crucial for memory efficiency and performance when dealing with large datasets.

```
1 # Helper function for batching
2 def batch_generator(data: List[Any], batch_size: int) → Generator[List[Any], None, None]:
3     """Yields successive n-sized chunks from a list."""
4     for i in range(0, len(data), batch_size):
5         yield data[i : i + batch_size]
```

Now, let's define the main class that puts everything together.

```
1
2 class FAQEngine:
3     """
4     An engine for setting up and querying a FAQ database using Qdrant and HuggingFace embeddings.
5     """
6     def __init__(self,
7                 qdrant_url: str = "http://localhost:6333",
8                 collection_name: str = "python-faq",
9                 embed_model_name: str = "nomic-ai/nomic-embed-text-v1.5"):
```

```

10
11     self.collection_name = collection_name
12
13     # Initialize the embedding model
14     print("Loading embedding model...")
15     self.embed_model = HuggingFaceEmbedding(
16         model_name=embed_model_name,
17         trust_remote_code=True
18     )
19
20     # Dynamically get the vector dimension from the model
21     self.vector_dim = len(self.embed_model.get_text_embedding("test"))
22     print(f"Embedding model loaded. Vector dimension: {self.vector_dim}")
23
24     # Initialize the Qdrant client
25     self.client = QdrantClient(url=qdrant_url, prefer_grpc=True)
26     print("Connected to Qdrant.")
27
28     @staticmethod
29     def parse_faq(text: str) → List[str]:
30         """Parses the raw FAQ text into a list of Q&A strings."""
31         return [
32             qa.replace("\n", " ").strip()
33             for qa in text.strip().split("\n\n")
34         ]
35
36     def setup_collection(self, faq_contexts: List[str], batch_size: int = 64):
37         """
38         Creates a Qdrant collection (if it doesn't exist) and ingests the FAQ data.
39         """
40         # Check if collection exists, create if not

```

```
41     try:
42         self.client.get_collection(collection_name=self.collection_name)
43         print(f"Collection '{self.collection_name}' already exists. Skipping creation.")
44     except Exception:
45         print(f"Creating collection '{self.collection_name}' ...")
46         self.client.create_collection(
47             collection_name=self.collection_name,
48             vectors_config=models.VectorParams(
49                 size=self.vector_dim,
50                 distance=models.Distance.DOT
51             )
52         )
53
54     print(f"Embedding and ingesting {len(faq_contexts)} documents...")
55
56     # Process data in batches
57     for batch in tqdm(batch_generator(faq_contexts, batch_size),
58                       total=(len(faq_contexts) // batch_size) + 1,
59                       desc="Ingesting FAQ data"):
60
61         # 1. Get embeddings for the batch
62         embeddings = self.embed_model.get_text_embedding_batch(batch, show_progress_bar=False)
63
64         # 2. Create Qdrant points with unique IDs and payloads
65         points = [
66             models.PointStruct(
67                 id=str(uuid.uuid4()), # Generate a unique ID for each point
68                 vector=vector,
69                 payload={"context": context}
70             )
71             for context, vector in zip(batch, embeddings)
```

```

72         ]
73
74     # 3. Upload points to the collection
75     self.client.upload_points(
76         collection_name=self.collection_name,
77         points=points,
78         wait=False # Asynchronous upload for speed
79     )
80
81     print("Data ingestion complete.")
82     print("Updating collection indexing threshold...")
83     self.client.update_collection(
84         collection_name=self.collection_name,
85         optimizer_config=models.OptimizersConfigDiff(indexing_threshold=20000)
86     )
87     print("Collection setup is finished.")
88
89     def answer_question(self, query: str, top_k: int = 3) → str:
90         """
91         Searches the vector database for a given query and returns the most relevant contexts.
92         """
93         # 1. Create an embedding for the user's query
94         query_embedding = self.embed_model.get_query_embedding(query)
95
96         # 2. Search Qdrant for the most similar vectors
97         search_result = self.client.search(
98             collection_name=self.collection_name,
99             query_vector=query_embedding,
100             limit=top_k,
101             score_threshold=0.5 # Optional: filter out less relevant results
102         )

```

```

103
104     # 3. Format the results into a single string
105     if not search_result:
106         return "I couldn't find a relevant answer in my knowledge base."
107
108     relevant_contexts = [
109         hit.payload["context"] for hit in search_result
110     ]
111
112     # Combine the contexts into a final, readable output
113     formatted_output = "Here are the most relevant pieces of information I found:\n\n---\n\n".join(relevant_contexts)
114     return formatted_output

```

The constructor, **init**, is responsible for setting up all the necessary components. It initializes the `HuggingFaceEmbedding` model from `llama-index`, which will handle the conversion of text to vectors.

A key detail here is that we dynamically determine the **vector_dim** by embedding a test string; this makes our code robust and adaptable to different embedding models. Finally, it establishes a connection to our Qdrant database. The print statements provide helpful feedback to the user when the script is run.

Before we can index our data, we need to clean it up. The **parse_faq** static method is a simple utility for this. It takes our raw `PYTHON_FAQ_TEXT` and splits it into a clean list of individual Question/Answer strings, which is the perfect format for our database.

The **setup_collection** method is where the indexing magic happens. It first checks if a Qdrant collection with our specified name already exists, preventing us from wastefully re-indexing data every time we start the application. If it doesn't exist, it creates a new one, configuring it with the correct vector size and distance metric.

Then, it begins the main ingestion loop, using our `batch_generator` helper to process the FAQ data in efficient chunks. Inside this loop, a three-step process occurs for each batch:

1. **Embed:** It converts the batch of text chunks into a list of numerical vectors.
2. **Structure:** It packages each vector into a Qdrant `PointStruct`, which includes a unique ID and a payload containing the original text. This payload is crucial, as it allows us to retrieve the human-readable context after a search.
3. **Upload:** It sends these points to the Qdrant collection.

Finally, the `answer_question` method is what our agent's tool will call. This is the "retrieval" step. It takes a user's query, converts it into a vector using the same embedding model, and then uses the `client.search` function to find the most similar vectors in our database.

It then extracts the original text from the payload of the top results and formats it into a single, clean string. This formatted text is the "context" that will ultimately be used to generate a high-quality answer.

Everything I've Written, One Button Away, With 40% Off  *Announcing My Ebooks Bundle + 50% Discount to my Followers*

7. Putting Everything Together and Testing the System

We have all the individual components: the server, the tools, and the RAG engine. Now, let's wire them together in `mcp_server.py` and launch our application.

```
1 # This should be at the end of the mcp_server.py file
2
3 if __name__ == "__main__":
4     # 1. Initialize our RAG Engine
5     print("Initializing FAQ Engine and setting up Qdrant collection...")
```

```
6     faq_engine = FAQEngine(qdrant_url=QDRANT_URL, collection_name=COLLECTION_NAME)
7
8     # 2. Ingest our data into the vector database
9     # This will create embeddings and store them
10    faq_engine.setup_collection(PYTHON_FAQ_TEXT)
11
12    # 3. Start the MCP server
13    print(f"Starting MCP server at http://{HOST}:{PORT}")
14    mcp_server.run()
```

This final block does three things in order:

1. Creates an instance of our FAQEngine.
2. Calls setup_collection() to process our text and load it into Qdrant.
3. Starts the mcp_server, which will now listen for incoming requests.

It is important to make sure your Docker container for Qdrant is still running. Then, open your terminal in the project directory and run the server:

```
1 python mcp_server.py
```

You should see output indicating that the FAQ Engine is being set up and that the server is running:

```
1 Initializing FAQ Engine and setting up Qdrant collection...
2 Collection not found. Creating and indexing...
3 Indexing complete.
```

```
4 Starting MCP server at http://127.0.0.1:8080
5 INFO:      Started server process [12345]
6 INFO:      Waiting for application startup.
7 INFO:      Application startup complete.
8 INFO:      Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
```

The server is live! Now, let's act as a client and send it requests. Open a **new terminal window** and use curl to interact with our agent.

Test Case 1: Querying the Python FAQ

Let's ask a question that should be answered by our internal knowledge base.

```
1 curl -X POST http://127.0.0.1:8080/mcp \
2 -H "Content-Type: application/json" \
3 -d '{
4     "messages": [
5         {
6             "role": "user",
7             "content": "Can you explain list comprehensions in Python?"
8         }
9     ]
10 }'
```

The agent should correctly choose the `python_faq_retrieval_tool`. The tool will query Qdrant, and you will get a response containing the relevant context from your `PYTHON_FAQ_TEXT`. The response from the server will be a JSON object that includes the tool call and the result:

```
1  {
2    "messages": [
3      // ... original message ...
4      {
5        "role": "assistant",
6        "tool_calls": [
7          {
8            "id": "call_abc123",
9            "type": "function",
10           "function": {
11             "name": "python_faq_retrieval_tool",
12             "arguments": "{\"query\": \"list comprehensions in Python\"}"
13           }
14         }
15       ]
16     },
17     {
18       "role": "tool",
19       "tool_call_id": "call_abc123",
20       "name": "python_faq_retrieval_tool",
21       "content": "Retrieved Context from FAQ:\nWhat is a list comprehension in Python?\nA: A list comprehension is a concise way to create lists in Python. It allows you to filter and transform data in a single line of code."
22     }
23   ]
24 }
```

Test Case 2: Querying the Web

Now, let's ask something our FAQ doesn't know. Make sure you have your **FIRECRAWL_API_KEY** in the .env file.

```
1 curl -X POST http://127.0.0.1:8080/mcp \
2 -H "Content-Type: application/json" \
3 -d '{
4   "messages": [
5     {
6       "role": "user",
7       "content": "What is the new Polars DataFrame library?"
8     }
9   ]
10  }'
```

This time, the agent will see that "Polars" isn't in its FAQ tool's docstring. It will instead choose the `web_search_tool`. The response will look similar but will contain the markdown content fetched by FireCrawl.

Congratulations! You have successfully built a sophisticated agentic RAG application from the ground up. By leveraging the Model Context Protocol, you've created a system that is modular, intelligent, and resourceful.

We've seen how MCP allows us to define distinct tools and how the agent uses simple docstrings to intelligently decide which capability to use for a given query. This powerful pattern allows you to create applications that can reason about their own abilities and dynamically pull information from private knowledge bases (via RAG) and the public internet, leading to far more accurate and helpful responses.

From here, you could expand the system by adding more tools (e.g., a calculator, a database query tool), ingesting more complex documents into your RAG pipeline, or connecting it to a real user interface. The MCP-powered foundation you've built is ready for whatever you want to build next.

If you like the article and would like to support me, make sure to:

- 🖐️ Clap for the story (50 claps) to help this article be featured
- Subscribe to [the Data & Beyond](#)  Newsletter
- Follow me on [Medium](#) 
- 📄 View more content on my [Medium profile](#) 
- 🔔 Follow Me: [LinkedIn](#)  | [Youtube](#)  | [GitHub](#)  | [Twitter](#) 

Subscribe to my newsletter, To Data & Beyond, to get full and early access to my articles:

[To Data & Beyond](#) | [Youssef Hosni](#) | [Substack](#)  *Data Science, Machine Learning, AI, and what is beyond them. Click to read To Data & Beyond, by Youssef Hosni, a...*

Are you looking to start a career in data science and AI, and do not know how? I offer data science mentoring sessions and long-term career mentoring:

- Mentoring sessions: <https://lnkd.in/dXeg3KPW> 
- Long-term mentoring: <https://lnkd.in/dtdUYBrM> 

Let's have a 1:1 call



- 1:1 Free Mentorship
- Data Science Career Planning
- Data Science Project Review

 topmate.io/youssef_hosni



By **Freedium.** – *made with care.*

[About](#)

[Privacy](#)

[Terms](#)

[RSS](#)

[GitHub](#)

[Codeberg](#)

